

APUNTESWEB · INGENIERÍA INFORMÁTICA

BASES DE DATOS

Segundo curso · 1.º cuatrimestre · 9 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. MODELO ENTIDAD RELACIÓN

Escrito por **Adrián Quiroga Linares**. [adriangl5](#)

El **Modelo Entidad Relación** es una técnica de diseño de **arriba a abajo**.

1.1 Diseño de una Base de Datos.

El proceso de diseño se divide en 4 fases:

- **Fase Inicial:** el diseñador debe realizar una **caracterización de las necesidades de datos** de los posibles usuarios. Debe relacionarse con expertos y usuarios del dominio para realizar la tarea. Y así obtenemos una **especificación de requisitos** del usuario.
- **Fase de diseño conceptual:** se escoge el modelo de datos y traducimos los requisitos en un **esquema conceptual de base de datos**. Este esquema realizado en esta fase de **diseño conceptual** proporciona una visión general de la empresa. En este esquema representamos: **Conjuntos de Entidades, Conjuntos de Relaciones, Atributos y Restricciones**.
- **Fase de especificación de requisitos funcionales:** se describen los tipos de operaciones o **transacciones** que se llevarán a cabo sobre los datos.
- **Fase final:**
 - **Fase de Diseño-Lógico:** Traduce el esquema Conceptual en un esquema de Relación.
 - **Fase de Diseño-Físico,** se especifican las características físicas de la base de datos. El esquema físico es fácilmente modificable, sin embargo las del **esquema lógico mas complejo de modificar**.

Transacción: es la descripción de un operación a llevar a cabo sobre los datos almacenados en la BD. Puede implicar tanto la creación, modificación, consulta o eliminación de los datos.

Se pueden obtener varios diseños completos y sin redundancia. Sin embargo se puede decir que tiene mayor calidad aquel que restrinja al máximo las opciones que no se deberían poder realizar.

Debemos evitar dos peligros:

- **Redundancia:** es importante no repetir información para tener un buen diseño. El mayor problema de la información repetida es que al actualizar la información, puede quedar alguna copia sin actualizar y se podría confundir todo.
- **Incompletitud:** puede hacer que sea muy difícil modelar algunos aspectos.

1. 2 Modelo Entidad Relación

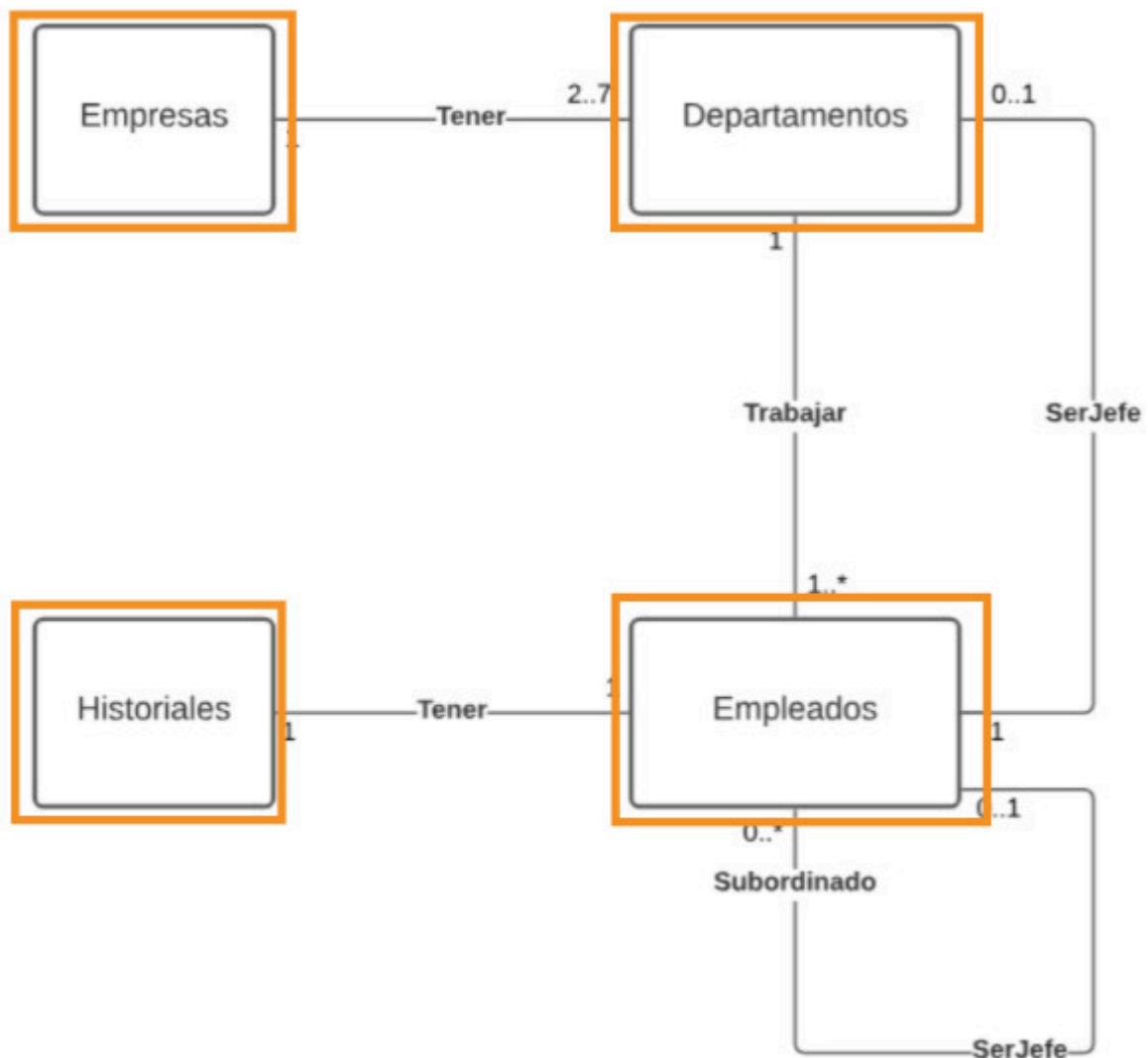
Emplea 3 conceptos básicos, los conjuntos de entidades, los conjuntos de relaciones y los atributos. Tiene asociada una representación en forma de diagramas.

1.2.1 Conjuntos de Entidades

Una **entidad** es una cosa/objeto del mundo real distinguible. Esta entidad es identificable de forma **unívoca**. Pueden ser concretas (*Personas*) o abstractas (*Reserva de un vuelo*).

Un **Conjunto de Entidades** es un grupo de objetos con existencia física o conceptual (*entidades*).

La **Extensión** de un conjuntos de entidades se refiere a la colección real de entidades que pertenecen al conjunto de entidades en cuestión.



1.2.2 Conjuntos de Relaciones

Una **Relación** es una asociación entre varias **entidades**. La relación *Tutor*, asocia a la entidad *profesor* con la entidad *alumno*.

Un **Conjuntos de Relaciones** es un conjunto de relaciones del mismo tipo (*2 entidades mínimo*).

La asociación entre conjuntos de entidades se conoce como **participación**. Los conjuntos de entidades $E1, E2 \dots, En$ participan en el conjunto de relaciones R .

La función que desempeña una entidad en una relación se denomina **rol**. Los roles suelen ser implícitos y no se especifican. Sin embargo, si en una relación intervienen entidades de un **mismo conjunto** desempeñarían diferentes roles. Este caso se denomina **Conjunto de Relaciones Recursivo**.

El conjunto de relaciones es **binario** si, involucra a solo 2 conjuntos de entidades. Si involucra a 3 **terciario**, etc. El número de conjuntos de entidades que participan en un conjunto de relaciones es el **grado** de ese conjunto de relaciones.

1.2.3 Atributos

Son **propiedades** de las entidades y las relaciones de interés para la misión de la base de datos, solo se incorpora a la BD aquellos **atributos** que sean **necesarios**.

Para cada atributo hay un conjuntos de valores permitidos llamado **dominio**.

- **Atributos simples** (*DNI*)
- **Atributos compuestos** (formados a partir de varios simples) (*fecha*)
- **Monovalorados** (solo toman un valor)
- **Multivalorados** (pueden tener un conjunto de valores) (*teléfono*, una persona puede tener varios números de teléfono)
- **Derivados** (su valor se puede obtener a partir del valor de otros atributos o entidades relacionados, el valor de estos atributos **nunca se almacena**). Los atributos además pueden tomar valores nulos.

1.3 Restricciones

Las **restricciones** determinan la semántica de la BD, es decir, su comportamiento. Una base de datos con los mismo conjuntos de entidades y relaciones será diferente si sus restricciones son diferentes.

La **multiplicidad** es el rango de Entidades de un Conjunto de Entidades que puedes relacionarse mediante un **Conjunto de Relaciones**, con una Entidad de otro Conjunto de Entidades. El máximo de

este rango es la **Cardinalidad** y el mínimo la **Participación**.

1.3.1 Correspondencia de cardinalidades

Expresa el **número de entidades** a las que otra entidad se puede asociar mediante un conjunto de relaciones.

- **Uno a uno:** Cada entidad de A se asocia, a lo sumo con una de B y viceversa.
- **Uno a varios:** cada entidad de A se asocia con cualquier número de entidades de B . Cada entidad de B se puede asociar, a lo sumo, con una de A .
- **Varios a uno:** cada entidad de A se asocia a lo sumo con una de B . Sin embargo, cada entidad de B se puede asociar con cualquiera de A .
- **Varios a varios:** cada entidad de A se asocia con cualquiera de B y viceversa

1.3.2 Restricciones de Participación Se dice que la **participación** de un conjunto de entidades E en un conjunto de relaciones R es **total** si cada entidad de E participa, al menos en una relación de R .

Si solo **algunas** entidades de E participan en relaciones de R , se dice que la participación del conjunto E en R es **parcial**.

1.3.3 Claves

Clave Primaria: Todo conjunto de entidades debe tener un **clave primaria**. Una **clave primaria** es un conjunto mínimo de atributos que tienen valores diferentes para cada entidad en cada conjunto de entidades. La clave primaria debe mantener su condición de valores **diferentes** en el pasado, presente y futuro. La estructura de las **claves primarias** de un conjunto de relaciones depende de la **correspondencia de cardinalidad** del conjunto.

Super Clave: conjunto de atributos que diferencia a una **entidad** del resto. **Clave Candidata:** posible **clave primaria**. Para escoger la **clave primaria** debemos escoger de entre todas las **claves candidatas** la **minimal** (la que esté formada por menor número de atributos).

1.4 Atributos Redundantes

Cada atributo que describe una propiedad de un conjunto, **está sólo una vez** en el Modelo, y además está en el conjunto de entidades o en el de relaciones al que pertenece. Si hace falta el atributo en otro lugar se obtiene a través de una **relación**.

1.5 Diagramas Entidad-Relación

Otro concepto nuevo es el de **conjunto de entidades fuertes o débiles**, las **débiles** necesitan la existencia de un conjunto de entidades **fuertes**. Se dice **conjunto de entidades débiles** a aquel conjunto de entidades que no tiene atributos suficientes que le permitan formar una **clave primaria**. Si tienen clave primaria son **conjuntos de entidades fuertes**.

La relación que asocia a conjuntos de entidades fuertes y débiles se llama **relación identificadora**.

Aunque los conjuntos de entidades débiles no tienen clave primaria, hace falta un medio para distinguir entre todas las entidades del conjunto de entidades débiles que dependen de una entidad fuerte concreta. El **discriminador** (*clave parcial*) de un conjunto de entidades débiles es un conjunto de atributos que permite que se haga esta distinción.

1.6 Modelo Entidad-Relación Extendido

Los conjuntos de entidades pueden incluir **subgrupos** de entidades que se diferencian de alguna forma de las demás entidades del conjunto. Las **subclases** tienen las mismas entidades que las superclases pero **agrupadas** en base al cumplimiento de alguna característica. Este proceso de establecimiento de subgrupos dentro del conjunto de entidades se denomina **especialización**.

Solemos decir que existe una **jerarquía** entre este grupo que engloba a los otros **subgrupos**. Por ejemplo, tenemos al conjunto *personas* sin embargo lo podemos desglosar en *alumnos* y *profesores*.

Tipos de jerarquías:

- **Solapada total:** todas las entidades deben pertenecer a un subconjunto y alguna puede pertenecer a varios.
- **Solapada parcial:** algunas entidades pueden no pertenecer a ningún subconjunto y alguna puede pertenecer a varios.
- **Disjunta total:** todas las entidades deben pertenecer a un subconjunto y solo puede aparecer en ese subconjunto.
- **Disjunta parcial:** algunas entidades pueden no pertenecer a ningún subconjunto pero las que aparecen no pueden estar presentes en subconjuntos diferentes.

Además existe **herencia de atributos**, los atributos de la superclase son heredados por los de la subclases. Las subclases además **heredan la participación** en los conjuntos de relaciones en los que participa su superclase

La **generalización** es la relación de contención que existe entre el conjunto de entidades de nivel superior y uno o varios de nivel inferior. Para crear **generalizaciones** los **atributos deben tener un nombre común** y representarse mediante la entidad de nivel superior.

Se puede entender como el proceso contrario a la **especialización**. En la especialización se "segmenta" una **superclase** y en la generalización se "juntan" varias **subclases** para construir una **superclase**.

Además podemos diferenciar entre **jerarquía** y **retículo**, en la jerarquía un conjunto de entidades dado sólo puede estar implicado como conjunto de entidades de nivel inferior en una relación, es decir, tienen **herencia única**. **Retículo**: si un conjunto de entidades es de nivel inferior en más de una relación tiene **herencia múltiple** y se denomina retículo.

Restricciones a las generalizaciones:

- **Definida por el atributo (o por la condición):** la pertenencia a una subclase se evalúa en función del cumplimiento de una condición, es decir, se evalúan en función del mismo atributo.
- **Definida por el usuario:** las subclases no están restringidas por una condición de pertenencia, si no que el usuario asigna las entidades a un conjunto dado.
- **Restricción de completitud:** especifica si una entidad de la superclase debe pertenecer al menos a unos de los conjuntos (*total*) o no (*parcial*).

A mayores contamos con la **Agregación**: Las agregaciones generan conjuntos de entidades de nivel conceptual superior, agrupando conjuntos de entidades relacionadas mediante conjuntos de **relaciones** en un concepto único. Su objetivo fundamental es poder establecer conjuntos de relaciones entre conjuntos de relaciones.

Se puede pensar como relacionar un **conjunto de relaciones con otro** para obtener así atributos de varios conjuntos de entidades, evitando aumentar el grado de las relaciones. Como máximo deberíamos tener relaciones de **grado 3 y después usar la agregación**.

DEFINICIONES

Entidad: Objeto, de un determinado tipo, identificable de forma unívoca.

Conjunto de Entidades: Grupo de objetos de un tipo, con las mismas propiedades, que se identifican en la Base de Datos como poseedores de una existencia independiente.

Relación: Asociación identificable de forma unívoca que incluye una Entidad de cada uno de los Conjuntos de Entidades participantes.

Conjuntos de Relaciones: Grupo de relaciones entre Entidades de Conjuntos de Entidades Participantes.

Grado de un Conjunto de Relaciones: número de Conjuntos de Entidades que participan en las Relaciones individuales del Conjunto de Relaciones.

Atributo: propiedad de un Conjunto de Entidades que participan en las Relaciones individuales del Conjunto de Relaciones.

Dominio de Atributo: Conjunto de valores permitido para el Atributo.

Multiplicidad: Rango (*número mínimo y número máximo*) de Entidades de un Conjunto de Entidades que puede relacionarse, mediante un Conjunto de Relaciones, con una Entidad de otro(s) Conjunto(s) de Entidades.

Participación: Determina si todas, o sólo parte de las Entidades de un Conjunto de Entidades que pueden relacionarse, mediante un Conjunto de Relaciones. Es el mínimo rango de la Multiplicidad.

Cardinalidad: Describe el número máximo de Entidades de un Conjunto de Entidades que pueden relacionarse una Entidad de otro(s) Conjunto(s) de Entidades a través de un Conjunto de relaciones. Es el máximos del rango de la Multiplicidad. **Correspondencia de Cardinalidades:** Cardinalidades correspondientes a los conjuntos de entidades participantes en un Conjunto de Relaciones.

Clave Candidata: conjunto mínimo de atributos que identifican de forma unívoca cada Entidad dentro de un Conjunto de Entidades.

Clave Primaria: Clave candidata seleccionada por el diseñador.

Conjunto de Entidades Fuerte: Conjunto de Entidades que no depende de ningún otro Conjunto de Entidades.

Conjunto de Entidades Débil: Conjunto de Entidades que depende existencialmente de otro Conjunto de Entidades.

Superclase: Conjunto de Entidades que incluye uno o más subgrupos diferentes en sus Entidades, los cuáles es preciso representar en el Modelo de datos.

Subclase: Subgrupo diferenciado de Entidades de un Conjunto de Entidades, que necesita ser representado en el Modelo de datos.

Herencia de Atributos en Jerarquías: Las Subclases de una jerarquía poseen los atributos de su Superclase (sin necesidad de indicarlo explícitamente).

Restricción de Participación: Determina si todo miembro de la Superclase debe participar, o no, como miembro de una Subclase.

Restricción de Disyunción: Indica si es posible que un miembro de una Superclase puede ser miembro de varias Subclases o sólo puede ser miembro de una Subclase.

Agregación: Conjunto de Entidades de nivel conceptual superior compuesto por un Conjunto de Relaciones entre Conjuntos de Entidades.

2. MODELO RELACIONAL

Escrito por **Adrián Quiroga Linares**.

2.1 Introducción

El **Modelo Relacional** es un enfoque para organizar datos utilizando **tablas**. Cada tabla es como una hoja de cálculo con **filas** y **columnas**, donde las columnas representan los **atributos** y las filas son **tuplas** (conjuntos de valores que describen una entidad). Por ejemplo, una tabla llamada *profesor* puede tener columnas como *ID*, *nombre*, *departamento*, y *sueldo*, donde cada fila representa un profesor.

En el contexto del modelo relacional, una **relación** es el nombre técnico para una tabla. Cada relación se puede ver como el resultado de combinar, mediante un **producto cartesiano**, todos los posibles valores de los atributos. Este concepto significa que, aunque en la práctica solo vemos filas específicas, una tabla podría, en teoría, contener todas las combinaciones posibles de los valores de sus atributos.

2.2 Estructura de las BD Relacionales

Una base de datos relacional consiste en un conjunto de **tablas**, a cada una de las cuales se le asigna un nombre **único**.

Una **Relación** es una **tabla** con filas y columnas resultado del producto cartesiano de conjuntos de elementos de interés.

Por norma general, una **fila** representa una asociación entre un conjunto de valores. Una relación entre n valores se representa como una **n-tupla** de valores, es decir como una **tupla** de n valores, que se corresponde con una **fila de una tabla**.

En el **MR**, el término **relación** se utiliza para referirse a una **tabla**, mientras el término **tupla** se utiliza para referirse a una **fila**. El término **atributo** se refiere a una **columna** de la tabla. El **Grado de una Relación** es el número de atributos que forman la relación.

La **Cardinalidad** es el número de **tuplas** que contiene la relación

Cada **atributo** tiene un conjunto de valores permitido, llamado **dominio** del atributo. Es necesario que todo dominio sea **atómico**. Es **atómico** si no es compuesto o multivalorado. El **MER** los permite.

2.3 Esquema de la BD Relacional

La definición de todas las tablas que forman una BD se denomina **esquema de la BD** (*el diseño lógico de la BD*). El esquema es la definición general, los datos particulares almacenados en un momento se denominan **ejemplar de la BD**. En lenguaje de programación el esquema se corresponde con el concepto de definición de tipos y el concepto de **relación** con el de variable. En general, los esquemas de las relaciones consisten en una lista de los atributos y de sus dominios correspondientes.

2.4 Claves

Cada **tupla** dentro de una tabla debe ser distinguible de todas las restantes tuplas de esa tabla, por tanto toda tabla deberá tener un **conjunto de atributos** que esté garantizado que tienen **valores diferentes en cada una de las filas de la tabla**. Esta unicidad debe cumplirse para las filas que estuvieron almacenadas en la tabla en el pasado, para las filas que están almacenadas en el presente y para cualquier fila que se pueda almacenar en el futuro.

Superclave: es un conjunto de uno o varios atributos que, considerados conjuntamente, permiten identificar de manera unívoca una **tupla** de la relación.

Claves Candidatas: son aquellas superclaves mínimas (*no se puede eliminar ningún atributo sin perder la condición de superclave*).

Clave primaria: es una de las claves candidatas, normalmente se escoge la más estable en el tiempo.

El modelo relacional incorpora **redundancia controlada** para visualizar las relaciones conceptuales entre diferentes tablas. Esta redundancia controlada se plasma incorporando a una tabla atributos denominados **clave externa** que hacen referencia a claves candidatas de otras tablas.

Los valores de **clave externa** están restringidos a valores que existan previamente en la clave candidata referenciada, generando lo que se conoce como **restricción de integridad referencial**.

2.5 Diagramas de esquema

No hay definido un modelo concreto de representación, cualquier forma es válida siempre que deje muy claro cuáles son las **claves externas** y a qué claves candidatas hacen referencia.

La gran diferencia entre el **MER** y el **MR** es la existencia o no de **redundancia de atributos**. En el **Modelo Relacional** se representa explícitamente una cierta **redundancia controlada** mediante las claves externas y la integridad referencial. En el **Modelo Entidad-Relación**, simplemente **no puede haber redundancia**.

DEFINICIONES

Relación: tabla con filas y columnas resultado del producto cartesiano de conjuntos de elementos de interés.

Atributo: Cada columna, con nombre diferente de una relación.

Dominio: Conjunto de valores permitidos para un atributo.

Tupla: Cada fila de una relación.

Cardinalidad: Número de tuplas que continúe la relación (*tabla*).

Base de Datos Relacional: Colección de relaciones, cada una con un nombre distintivo.

Esquema de Relación: Estructura de una relación definida por un conjunto de parejas de atributos y sus correspondientes dominios.

Esquema de la Base de Datos Relación: Conjunto de esquemas de relaciones, cada uno con un nombre distintivo.

Superclave: Atributo o conjunto de atributos que identifica(n) de forma unívoca cada tupla dentro de una relación.

Clave Candidata: Superclave tal que ningún subconjunto propio de la misma es superclave de la relación (*superclave mínima*).

Clave Primaria: Clave candidata seleccionada por el diseñador de la Base.

Clave Externa: Atributo o conjunto de atributos dentro de una relación que se corresponde(n) con una clave candidata de alguna relación (incluso la misma) de la Base de Datos.

Valor Null: Valor desconocido de un atributo en una tupla de una relación.

Integridad de Entidad: En una relación, ningún atributo de una clave candidata puede tomar valor null.

Integridad Referencial: Si hay una clave externa en una relación, el valor de dicha clave externa debe corresponder al valor de la clave candidata relacionada en la relación de origen o tomar valor null.

3. Transformación de MER a MR

Escrito por **Adrián Quiroga Linares**.

TIP

Solapada -> incluir discriminadores binarios Parcial-> incluir una tabla para la superclase a mayores de las subclases Total-> no hay tabla para la superclase Disjunta->no hay discriminadores binarios

Ejemplo

4. Lenguajes Formales de Consulta

Escrito por **Adrián Quiroga Linares**.

4.1 Introducción

Para realizar una implementación operativa del **modelo relacional** es necesario que exista un lenguaje que permita crear, modificar y eliminar las tablas del **esquema relacional** de la BD y, una vez creadas las tablas, poder añadir tuplas con la información adecuada y obtener información a partir de los datos almacenados.

Los lenguajes de consulta pueden clasificarse en **procedimentales** o **no procedimentales**, y tienen 5 operaciones básicas (*selección, proyección, producto cartesiano, unión de conjuntos y diferencia de conjuntos*) a partir de las cuales se puede hacer cualquier otra operación a realizar.

- Los **lenguajes procedimentales**: en ellos el usuario indica al sistema que lleve a cabo una **serie de operaciones** en la base de datos para calcular el resultado deseado.
- Los **lenguajes no procedimentales**: en ellos el usuario describe la información deseada **sin establecer** un procedimiento concreto para obtener esa información.

4.2 Álgebra Relacional

El **álgebra relacional** es un lenguaje de consulta **procedimental** que permite operar sobre relaciones (tablas) en bases de datos relacionales. Define un conjunto de operaciones que, operando sobre tablas, devuelve tablas.

Las operaciones pueden ser **unarias** (*operan sobre una única tabla*) o **binarias** (*operan sobre 2 tablas*).

Operación Selección (σ)

Filtra tuplas que satisfacen una condición (*unaria*). Selección $\rightarrow \sigma$ Predicado $\rightarrow$ subíndice Relación $\rightarrow$ entre paréntesis

$$\sigma_{\text{suelto} > 90000}(\text{profesor})$$

Operación Proyección (Π)

Determina qué **atributos** visualizar y cuáles se eliminan de los resultados (*unaria*). Proyección $\rightarrow \Pi$ Atributos a mostrar $\rightarrow$ subíndice Relación $\rightarrow$ entre paréntesis

$$\Pi_{ID,nombre,sueldo}(profesor)$$

Composición de operaciones

Las operaciones relacionales se pueden **componer** para realizar consultas más complejas. Por ejemplo, si queremos obtener los nombres de todos los profesores del departamento de Física:

$$\Pi_{nombre}(\sigma_{nombre_dept='Física'}(profesor))$$

Operación Unión ($\cup$)

La unión combina todas las tuplas de dos relaciones **compatibles**, eliminando duplicados. Para que dos relaciones sean compatibles, deben tener el mismo número de atributos y los mismos dominios (tipos de datos).

$$\Pi_{asignatura_id}(\sigma_{semestre='Otoño' \wedge año=2009}(sección)) \cup \Pi_{asignatura_id}(\sigma_{semestre='Primavera' \wedge año=2010}(sección))$$

Operación Diferencia de Conjuntos ($-$)

La diferencia de conjuntos devuelve las tuplas que están en la primera relación pero no en la segunda.

$$\Pi_{asignatura_id}(\sigma_{semestre='Otoño' \wedge año=2009}(sección)) - \Pi_{asignatura_id}(\sigma_{semestre='Primavera' \wedge año=2010}(sección))$$

Intersección de conjuntos ($\cap$) (Inner Join)

La intersección devuelve las tuplas que están presentes tanto en una tabla como en otra.

$$\Pi_{asignatura_id}(\sigma_{semestre='Otoño' \wedge año=2009}(sección)) \cap \Pi_{asignatura_id}(\sigma_{semestre='Primavera' \wedge año=2010}(sección))$$

Reunión Externa

La **reunión externa** es una variación de la reunión natural que preserva las tuplas de una o ambas tablas, incluso si no encuentran coincidencias. Rellena los valores faltantes con `NULL`.

Existen tres tipos de reuniones externas:

1. **Reunión externa por la izquierda ($\bowtie$)**: preserva todas las tuplas de la relación izquierda.
2. **Reunión externa por la derecha ($\bowtie$)**: preserva todas las tuplas de la relación derecha.
3. **Reunión externa completa ($\bowtie$)**: preserva todas las tuplas de ambas relaciones. Equivalente a la unión más la intersección

Producto Cartesiano ($\times$) Comprime información procedente de dos relaciones y crea una relación cuyo nº de atributos es la suma de atributos de las 2 relaciones combinadas y su nº de tuplas es el producto de las tuplas de ambas relaciones (*binaria*).

Reunión Natural ($\bowtie$)

La **reunión natural** (natural join) combina dos relaciones pero solo conserva las tuplas donde los valores de los atributos comunes coinciden. Es como un producto cartesiano seguido de una selección y proyección para mantener solo las coincidencias en los atributos comunes.

Reunión Zeta ($\bowtie_{\theta}$)

La **reunión zeta** (o theta join) es una operación similar a la reunión natural, pero permite especificar una condición arbitraria entre las tuplas, no necesariamente de igualdad en los atributos comunes.

Supongamos que queremos combinar **Profesor** y **Curso** en función de una condición arbitraria, por ejemplo, que el **departamento** de **Profesor** contenga la palabra "Informática". Esto se expresa como:

$$Profesor \bowtie_{departamento="Informática"} Curso$$

Operación Renombramiento

El renombramiento se utiliza para cambiar los nombres de las relaciones o atributos, lo cual es útil para evitar ambigüedades en operaciones como el producto cartesiano. Se denota con la letra griega ρ .

- **Ejemplo:** Si queremos renombrar la relación `profesor` como `P`:

$$\rho_P(profesor)$$

Asignación ($\leftarrow$)

La **asignación** permite almacenar temporalmente el resultado de una consulta en una variable, lo cual es útil para reutilizar ese resultado en operaciones posteriores.

Ejemplo: Supongamos que queremos obtener el producto cartesiano de **Profesor** y **Enseña** y luego hacer una selección sobre el resultado.

1. Asignamos el producto cartesiano a una variable:

$$temp1 \leftarrow Profesor \times Enseña$$

2. Ahora, hacemos una selección sobre **temp1** para encontrar las coincidencias en **ID**:

$$temp2 \leftarrow \sigma_{Profesor.ID=Enseña.ID}(temp1)$$

3. Por último, podemos proyectar solo los nombres y cursos:

$$resultado \leftarrow \Pi_{nombre, curso_id}(temp2)$$

Este mecanismo ayuda a descomponer consultas complejas en pasos más simples y organizados.

4.3 Operaciones del Álgebra Relacional Extendida

Proyección Generalizada

La proyección generalizada es una extensión de la proyección simple. En la proyección tradicional, se seleccionan columnas de una tabla sin aplicar transformaciones o cálculos sobre los datos. Sin embargo, la **proyección generalizada** permite incluir operaciones aritméticas o de manipulación de cadenas en los atributos seleccionados. Esto es útil para generar nuevos valores a partir de los datos existentes.

$$\Pi_{ID, nombre, nombre_dept, sueldo \div 12}(\text{profesor})$$

Agregación

La operación de **agregación** permite aplicar funciones como suma, promedio, conteo, mínimo y máximo a conjuntos de valores. Estas funciones devuelven un único valor a partir de una colección de valores, y son útiles cuando se necesitan realizar cálculos estadísticos sobre los datos.

1. **Suma sum**: Devuelve la suma de los valores.

◦ Ejemplo:

$$\text{sum}(\{1, 1, 3, 4, 4, 11\}) = 24$$

2. **Promedio avg**: Devuelve la media de los valores.

◦ Ejemplo:

$$\text{avg}(\{1, 1, 3, 4, 4, 11\}) = 4$$

3. **Conteo count**: Devuelve el número de elementos en la colección.

◦ Ejemplo:

$$\text{count}(\{1, 1, 3, 4, 4, 11\}) = 6$$

4. **Mínimo min**: Devuelve el valor mínimo.

◦ Ejemplo:

$$\min(\{1, 1, 3, 4, 4, 11\}) = 1$$

5. **Máximo max:** Devuelve el valor máximo.

◦ Ejemplo:

$$\max(\{1, 1, 3, 4, 4, 11\}) = 11$$

Resumen

- **Proyección generalizada:** permite realizar cálculos o concatenaciones sobre las columnas de una relación, como en el ejemplo de dividir el sueldo anual por 12 para obtener el sueldo mensual.
- **Agregación:** permite aplicar funciones como `sum`, `avg`, `count`, `min`, y `max` para resumir datos en una relación. También permite agrupar los datos por ciertos atributos antes de aplicar la función de agregación, como calcular el promedio por departamento.

4.5 Cálculo Relacional de Tuplas

El **cálculo relacional de tuplas** es un lenguaje de consultas **no procedimental**** que describe la información deseada sin especificar un procedimiento concreto para obtenerla. Las consultas se expresan como:

`{t | P(t)}`

Esto representa el conjunto de todas las tuplas `t` tales que el predicado `P` es cierto para `t`.

`{t | t ∈ profesor ∧ t[sueldo] > 80000}`

El cálculo relacional de tuplas es equivalente en potencia expresiva al álgebra relacional básico (sin operaciones relacionales extendidas). Esto significa que todas las expresiones del álgebra relacional pueden representarse en el cálculo relacional de tuplas y viceversa.

4.6 Cálculo Relacional de Dominios

Lenguaje **no procedimental**.

Una segunda forma de cálculo relacional, denominada **cálculo relacional de dominios**, usa **variables de dominio**, que toman sus valores del **dominio de un atributo**, en vez de hacerlo para una tupla completa. El cálculo relacional de dominios, no obstante, se halla estrechamente relacionado con el cálculo relacional de tuplas. El cálculo relacional de dominios sirve de base teórica al ampliamente utilizado lenguaje QBE (véase Apéndice C.1 en red), al igual que el álgebra relacional sirve como base para el lenguaje SQL.

Las expresiones del cálculo relacional de dominios son de la forma:

$$\{ \langle x_1, x_2, \dots, x_n \rangle \mid P \}$$

donde $x_1, x_2, \dots, x_n$ representan las variables de dominio. P representa una **fórmula compuesta por átomos**, como era el caso en el cálculo relacional de tuplas.

Un **lenguaje** que puede usarse para producir cualquier relación se denomina **relacionalmente completo**. Los tres explicados en este tema son relacionalmente completos.

5. Structured Query Lanjuaje (SQL), Tablas y Vistas

Escrito por **Adrián Quiroga Linares**.

SQL es el **lenguaje de consulta** de bases de datos más ampliamente utilizado. Aunque se le suele llamar lenguaje de consulta también tiene instrucciones para **crear, modificar y eliminar tablas, insertar modificar y eliminar tuplas y definir restricciones de integridad**.

5.1 Lenguaje de deficion de datos (LDD)

SQL proporciona comandos para la definición y modificación de esquemas de relación y el borrado de relaciones. El **LDD** permite la especificación de un conjunto de relaciones y de su información relativa, incluyendo:

- **El esquema de cada relación.**
- **El dominio de valores asociado a cada atributo.**
- **Las restricciones de integridad**
- **El conjunto de índices que se deben mantener para cada relación**
- **La información de seguridad y de autorización de cada relación**
- **La estructura de almacenamiento físico de cada relación en el disco.**

5.2 Tipos Básicos de Datos

- **char(n)**: Cadena de caracteres de longitud fija, donde (n) es especificado por el usuario. También se puede utilizar **character**.
- **varchar(n)**: Cadena de caracteres de longitud variable, con una longitud máxima de (n) especificada por el usuario. La forma completa **character varying** es equivalente.
- **int**: Número entero, un subconjunto finito de los enteros dependiente de la máquina. **integer** es equivalente.
- **smallint**: Entero pequeño, un subconjunto dependiente de la máquina del tipo de dominio entero.
- **numeric(p, d)**: Número de coma fija con precisión especificada por el usuario. Se compone de (p) dígitos (más el signo), de los cuales d pertenecen a la parte decimal. Por ejemplo, **numeric(3,1)** puede almacenar 44.5, pero no 444.5 ni 0.32.
- **real, double precision**: Números de coma flotante, con precisión dependiente de la máquina.
- **float(n)**: Número de coma flotante cuya precisión es, al menos, de (n) dígitos.

Todos los tipos incluyen un valor especial llamado **valor null** (nulo), que indica la ausencia de un valor.

Tipos de datos de fecha y hora:

- **date:** fecha del calendario que contiene el año, el mes y el día del mes
- **time:** hora del día en horas, minutos y segundos, se puede almacenar también el huso horario con `time with timezone`
- **timestamp:** combinación de date y time

5.3 Definición de tablas en SQL

Las relaciones se definen mediante el comando `CREATE TABLE`

```
CREATE TABLE departamento (  
    nombre_dept VARCHAR(20),  
    edificio VARCHAR(15),  
    presupuesto NUMERIC(12,2),  
    PRIMARY KEY (nombre_dept)  
);
```

Relación Base: Una tabla con nombre correspondiente a una relación del esquema de la Base de Datos dónde las tuplas están almacenadas físicamente en el Gestor de Bases de Datos.

1. **PRIMARY KEY** ($A_{j1}, A_{j2}, \dots, A_{jm}$):

- Los atributos forman la clave primaria de la relación.
- Los atributos deben ser no nulos y únicos.

2. **FOREIGN KEY** ($A_{k1}, A_{k2}, \dots, A_{kn}$) **REFERENCES** s:

- Indica que los valores de los atributos deben corresponder con los de la clave primaria de otra relación.
- Ejemplo: En la tabla `asignatura`, `foreign key (nombre_dept) references departamento`.
- Además si el nombre de ambos atributos que se relacionan no coinciden hay que especificarlo: `foreign key (nombre_dept) references departamento(nombre)`

```
CREATE TABLE sección (  
    asignatura_id VARCHAR(8),  
    secc_id VARCHAR(8),  
    semestre VARCHAR(6),  
    año NUMERIC(4,0),  
    edificio VARCHAR(15),
```

```
número_aula VARCHAR(7),
franja_horaria_id VARCHAR(4),
PRIMARY KEY (asignatura_id, secc_id, semestre, año),
FOREIGN KEY (asignatura_id) REFERENCES asignatura
);
```

5.4 Modificación de Tablas

Insertar una tupla:

```
INSERT INTO profesor (atrib1,...,atribn) VALUES (10211, 'Smith', 'Biología', 66000);
```

Eliminar todas las tuplas de una tabla:

```
DELETE FROM estudiante;
```

Eliminar relaciones tablas:

```
DROP TABLE profesor;
```

Modificar la estructura de tablas:

- **Añadir atributos:**

```
ALTER TABLE profesor ADD sueldo int;
...
```

– ****Eliminar atributos**:**

```
```sql
ALTER TABLE profesor DROP sueldo;
...
```

### # 5.5 Actualizaciones

**\*\*Actualizar todas las tuplas\*\***

```
```sql
UPDATE profesor SET sueldo = sueldo * 1.05;
```

5.6 Vistas

Son relaciones actuales que contienen el resultado de las consultas para hacer visible al usuario los datos que este puede ver. En SQL las vistas se definen con :

```
CREATE VIEW v AS (expresión de consulta);
```

Vista: El resultado dinámico de una o más operaciones relacionales sobre las relaciones base (y, opcionalmente, vistas previas) para producir otra relación. Una vista, como relación virtual, no tiene por qué existir físicamente en el Gestor de la Base de Datos, sino que puede producirse cuando se solicite por parte del usuario.

Razones para usar vistas:

1. **Seguridad:** Algunos datos deben permanecer ocultos para ciertos usuarios. Por ejemplo, un empleado puede necesitar ver el ID, nombre y departamento de un profesor, pero no su sueldo. Esto se logra mediante una consulta SQL que solo devuelve la información permitida.
2. **Personalización:** A veces, es útil crear relaciones que sean más intuitivas para los usuarios. Por ejemplo, se podría querer listar todas las secciones de asignaturas de un departamento específico en un semestre determinado. Para ello, se utiliza una consulta SQL que filtra la información necesaria.

Vistas Materializadas

Las **vistas materializadas** son una extensión de las vistas tradicionales en SQL que permiten almacenar los resultados de una consulta en la base de datos. Esto significa que, a diferencia de las vistas normales, que se recalculan cada vez que se accede a ellas, las vistas materializadas guardan un conjunto de datos que se puede consultar directamente.

1. **Mantenimiento Inmediato:** La vista se actualiza automáticamente cada vez que se modifica una de las tablas que la definen.
 2. **Mantenimiento Perezoso:** La vista se actualiza solo cuando se accede a ella, lo que puede resultar en datos desactualizados hasta que se consulte.
 3. **Actualización Periódica:** Algunas bases de datos permiten que la vista se actualice en intervalos programados, lo que puede llevar a que los datos estén desfasados.
- **Rendimiento Mejorado:** Almacenar resultados de consultas complejas o agregaciones puede acelerar significativamente el tiempo de respuesta, especialmente en aplicaciones que requieren respuestas rápidas.
 - **Reducción de Carga:** Las vistas materializadas permiten evitar la lectura de grandes conjuntos de datos subyacentes, al trabajar con un conjunto de datos más pequeño.

Si bien las vistas materializadas ofrecen ventajas en términos de rendimiento, también conllevan costos adicionales, como:

- **Almacenamiento:** Necesitan espacio adicional en disco para almacenar los resultados.
- **Sobrecarga de Actualización:** El mantenimiento de la vista puede requerir recursos y tiempo, especialmente si se actualiza con frecuencia.

La creación de tablas usando `CREATE TABLE ... AS` es similar a la creación de vistas (`CREATE VIEW`). La diferencia principal es que los datos en una tabla son copiados en el momento de la creación, mientras que una vista siempre refleja los resultados actualizados de la consulta que la define. Por ejemplo:

```
CREATE VIEW vista_profesores AS
SELECT * FROM profesor WHERE nombre_dept = 'Música';
```

Aquí, `vista_profesores` siempre mostrará los datos actuales de la tabla `profesor` donde `nombre_dept` es 'Música', pero no almacena esos datos de forma física como lo hace `t1`.

5.5 Restricciones de Integridad

Las **restricciones de integridad** garantizan la consistencia de la base de datos, especialmente durante las actualizaciones de información, evitando que las modificaciones realizadas por usuarios den lugar a una **pérdida de consistencia** de los datos.

Restricciones sobre una sola relación

1. **Not Null:** prohíbe la inserción de valores nulos para ese atributo, las claves primarias reciben esta restricción automáticamente.
2. **Unique:** indica que esos atributos forman una clave candidata, ningún par de tuplas, puede tener el mismo valor para esos atributos.
3. **Check(p):** especifica un predicado que deben satisfacer todas las tuplas de una relación.

```
CREATE TABLE profesor (
    ID INT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);

CREATE TABLE departamento (
    nombre_dept VARCHAR(50) UNIQUE,
    presupuesto DECIMAL(10, 2)
);

CREATE TABLE asignatura (
    asignatura_id INT PRIMARY KEY,
    nombre VARCHAR(100),
```



```
créditos INT CHECK (créditos > 0)
);
```

Integridad referencial

Se fuerza a que los valores válidos en las claves externas sean valores que existan en las claves candidatas referenciadas (*fundamentalmente para la consistencia interna de la base de datos*).

```
foreign key (nombre_dept) references departamento(nombre_dept)
```

SQL permite manejar violaciones de integridad referencial de distintas maneras. Las acciones más comunes son:

- **Cascade:** Si se borra o actualiza un registro en la tabla referenciada, el cambio se propaga a las tablas que referencian ese registro.

```
foreign key (nombre_dept) references departamento on delete cascade on update
cascade
```

- **Set null:** Al borrar o actualizar un registro referenciado, los valores de las columnas referenciadas se establecen a `null`.

```
foreign key (nombre_dept) references departamento(nombre_dept) on delete set
null
```

- **Set default:** Similar a `set null`, pero los valores se establecen a un valor predeterminado.

```
foreign key (nombre_dept) references departamento(nombre_dept) on delete set
default
```

Ejemplo SQL

```
-- Crear la tabla Categoría
CREATE TABLE Categoría (
    id_categoria INT PRIMARY KEY,
    nombre_categoria VARCHAR(100) NOT NULL
);

-- Crear la tabla Producto con clave foránea que utiliza ON DELETE SET NULL
CREATE TABLE Producto (
    id_producto INT PRIMARY KEY,
```

```

    nombre_producto VARCHAR(100) NOT NULL,
    id_categoria INT,
    FOREIGN KEY (id_categoria) REFERENCES Categoria(id_categoria)
    ON DELETE SET NULL
    ON UPDATE CASCADE
);

-- Crear la tabla Proveedor
CREATE TABLE Proveedor (
    id_proveedor INT PRIMARY KEY,
    nombre_proveedor VARCHAR(100) NOT NULL
);

-- Crear la tabla Pedido con clave foránea que utiliza ON DELETE CASCADE
CREATE TABLE Pedido (
    id_pedido INT PRIMARY KEY,
    id_proveedor INT NOT NULL,
    FOREIGN KEY (id_proveedor) REFERENCES Proveedor(id_proveedor)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

-- Crear la tabla Cliente
CREATE TABLE Cliente (
    id_cliente INT PRIMARY KEY,
    nombre_cliente VARCHAR(100) NOT NULL,
    pais_origen VARCHAR(50) DEFAULT 'Desconocido'
);

-- Crear la tabla Venta con clave foránea que utiliza ON DELETE SET DEFAULT
CREATE TABLE Venta (
    id_venta INT PRIMARY KEY,
    id_cliente INT NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente)
    ON DELETE SET DEFAULT
    ON UPDATE CASCADE
);

```

Acciones en violaciones

Cuando ocurre una violación de una restricción de integridad referencial, el sistema puede rechazar la transacción, revertir los cambios, o realizar alguna de las acciones mencionadas (cascade, set null,

etc.). Si no es posible resolver la violación con ninguna de estas acciones, se aborta la transacción.

Este conjunto de reglas asegura que las relaciones entre las tablas sean coherentes y se mantenga la integridad de los datos.

DEFINICIONES

Relación base: Una tabla con nombre correspondiente a una relación del esquema de la Base de Datos dónde las tuplas están almacenadas físicamente en el Gestor de Bases de Datos.

Vista: El resultado dinámico de una o más operaciones relacionales sobre las relaciones base (y, opcionalmente, vistas previas) para producir otra relación. Una vista, con relación virtual, no tiene por qué existir físicamente en el Gestor de la Base de Datos, sino que puede producirse cuando se solicite por parte del usuario.

6. Structured Query Language (SQL) Consultas

Escrito por **Adrián Quiroga Linares**.

6.1 Estructura Básica de una Consulta en SQL

Las consultas SQL tienen una estructura básica que consiste en tres cláusulas principales:

```
select atrib1,..., atribN  
from tabla  
where predicado
```

6.2 Select operación de proyección

Por defecto, SQL permite que se repitan valores en los resultados. Si varios profesores pertenecen al mismo departamento, podríamos obtener el nombre del departamento repetido. Si deseamos evitar estos duplicados, utilizamos la palabra clave `DISTINCT`:

```
SELECT DISTINCT nombre_dept  
FROM profesor;
```

SQL permite incluir operaciones matemáticas en las consultas. Por ejemplo, si queremos aumentar un 10% el sueldo de todos los profesores:

```
SELECT ID, nombre, nombre_dept, sueldo * 1.1  
FROM profesor;
```

También podemos renombrar tanto atributos como relaciones. Utilizamos la cláusula `AS` para renombrar columnas y tablas en las consultas SQL.

Cuando ejecutamos una consulta SQL, los resultados incluyen columnas cuyos nombres provienen de las tablas de la cláusula `FROM`. Sin embargo, si hay columnas con el mismo nombre en diferentes tablas, el resultado tendrá nombres duplicados. Además, cuando se utilizan expresiones en la cláusula `SELECT`, estas no tendrán un nombre explícito.

Si queremos que el atributo `nombre` aparezca como `nombre_profesor`, podemos modificar la consulta:

```
SELECT nombre AS nombre_profesor, asignatura_id
FROM profesor, enseña
WHERE profesor.ID = enseña.ID;
```

También podemos renombrar tablas utilizando la cláusula `AS`. Esto es especialmente útil cuando queremos hacer referencias a la misma tabla varias veces. Por ejemplo, si queremos comparar los sueldos de los profesores en un departamento específico:

```
SELECT DISTINCT T.nombre
FROM profesor AS T, profesor AS S
WHERE T.sueldo > S.sueldo AND S.nombre_dept = 'Biología';
```

Aquí, `T` y `S` son alias para la misma tabla `profesor`, permitiendo que la consulta sea más legible y que podamos diferenciar entre las dos instancias de la misma tabla.

6.3 Where operación de selección

Podemos usar la cláusula `WHERE` para aplicar condiciones. Por ejemplo, si queremos encontrar solo a los profesores del departamento de Informática cuyo sueldo sea mayor a 70,000 €:

```
SELECT nombre
FROM profesor
WHERE nombre_dept = 'Informática' AND sueldo > 70000;
```

Ejemplo de Uso de `BETWEEN` :

```
SELECT nombre
FROM profesor
WHERE sueldo BETWEEN 90000 AND 100000;
```

Ejemplo de Uso de `IN` :

```
SELECT nombre
FROM profesor
WHERE sueldo IN (900 ,1000);
```

Si queremos filtrar por un nulo

```
where profesor IS NULL
```

```
where profesor IS NOT NULL
```

6.4 Operaciones con Cadenas de Caracteres

SQL permite varias funciones que operan sobre cadenas de caracteres con `upper(s)` para convertir a mayúsculas y `lower(s)` para convertir a minúsculas, o `trim(s)` para eliminar espacios al final de las cadenas.

El operador `LIKE` se utiliza para buscar patrones en cadenas. Los caracteres especiales son `%` (cualquier subcadena) y `_` (cualquier carácter).

Ejemplo de Búsqueda de Patrones:

```
SELECT nombre_dept
FROM departamento
--Esto busca todos los departamentos cuyo nombre de edificio contenga "Watson"--
WHERE edificio LIKE '%Watson%';
--Esto busca todos los departamentos cuyo nombre de edificio termine por "Watson"--
WHERE edificio LIKE '%Watson';
```

NOTA

También se pueden hacer casts en SQL:

```
cast(entero as numeric(8,1))
```

6.5 Orden en la Presentación de las Tuplas

SQL permite controlar el orden de las filas devueltas con la cláusula `ORDER BY`. Por defecto, el orden es ascendente.

Ejemplo de Ordenación Ascendente:

```
SELECT nombre
FROM profesor
WHERE nombre_dept = 'Física'
ORDER BY nombre;
```

Orden Descendente y Múltiples Atributos:

```
SELECT *  
FROM profesor  
ORDER BY sueldo DESC, nombre ASC;
```

Esto ordena los resultados por `suel`do de manera descendente y, en caso de empate, por `nom`bre de manera ascendente.

6.6 Operaciones sobre Conjuntos

6.6.1 Unión (`UNION`)

Combina los resultados de dos consultas, eliminando duplicados por defecto. Si necesitas conservar los duplicados, usa `UNION ALL`.

```
SELECT columna FROM tabla1  
UNION  
SELECT columna FROM tabla2;
```

6.6.2 Intersección (`INTERSECT`)

Devuelve únicamente las filas comunes entre dos consultas, eliminando duplicados automáticamente.

```
SELECT columna FROM tabla1  
INTERSECT  
SELECT columna FROM tabla2;
```

6.6.3 Excepto (`EXCEPT`)

Devuelve las filas que están en la primera consulta pero no en la segunda. Para conservar duplicados, usa `EXCEPT ALL`.

```
SELECT columna FROM tabla1  
EXCEPT  
SELECT columna FROM tabla2;
```

Resumen

- **UNION**: Combina resultados y elimina duplicados.
- **INTERSECT**: Devuelve los elementos comunes.
- **EXCEPT**: Devuelve las filas exclusivas de la primera consulta.

6.7 Valor NULL

Los valores nulos en SQL son un concepto importante que implica ciertas peculiaridades en las operaciones aritméticas, de comparación y de conjunto. Aquí te explico en detalle cómo funcionan y cómo afectan las consultas:

```
SELECT r.A + 5 FROM tabla r;
```

Si `r.A` es nulo para alguna de las tuplas, el resultado de `r.A + 5` también será nulo para esa tupla. Esto se debe a que no se puede determinar un resultado numérico válido si uno de los operandos es nulo.

Las comparaciones que involucran valores nulos son problemáticas. Por ejemplo, consideremos la comparación:

```
1 < NULL
```

No se puede afirmar que esta comparación sea verdadera o falsa porque no se conoce el valor que representa `NULL`. Por lo tanto, SQL considera el resultado de cualquier comparación que involucre un valor nulo como `unknown` (desconocido). Esto crea un tercer estado lógico además de `true` (verdadero) y `false` (falso).

Debido a la introducción de `unknown`, las operaciones booleanas deben extenderse para incluir este estado adicional. Aquí están las reglas:

- **AND**

- `true AND unknown` → `unknown`
- `false AND unknown` → `false`
- `unknown AND unknown` → `unknown`

- **OR**

- `true OR unknown` → `true`
- `false OR unknown` → `unknown`
- `unknown OR unknown` → `unknown`

- **NOT**

- `NOT unknown` → `unknown`

6.8 Funciones de agregación

Las funciones de agregación toman un conjunto de valores y devuelven un único valor. SQL incluye las siguientes funciones de agregación incorporadas:

1. **Media (avg)**: Calcula la media aritmética de un conjunto de valores.
2. **Mínimo (min)**: Devuelve el valor mínimo de un conjunto.
3. **Máximo (max)**: Devuelve el valor máximo de un conjunto.
4. **Total (sum)**: Calcula la suma total de un conjunto de valores.
5. **Recuento (count)**: Devuelve el número de elementos en un conjunto.

```
SELECT AVG(sueldo) FROM profesor WHERE nombre_dept = 'Informática';
```

Cuando se trata de valores nulos, las funciones de agregación se comportan de la siguiente manera:

- **SUM** e **AVG** ignoran los valores nulos. Por ejemplo:

```
SELECT SUM(sueldo) FROM profesor;
```

Si algunas tuplas tienen **NULL** en **suelo**, estas se ignoran en el cálculo.

- **COUNT(*)** cuenta todas las tuplas, incluso si contienen valores nulos. Sin embargo, **COUNT(columna)** ignora los nulos en esa columna.

6.9 Agrupar Resultados

La cláusula **GROUP BY** unida a un **SELECT** **permite agrupar filas según las columnas que de indiquen como parámetros**. Se suele usar para obtener datos resumidos y agrupados por las columnas que se necesiten.

```
SELECT EmployeeID, COUNT(*) AS TotaPedidos, COUNT(ShipRegion) AS FilasNoNulas,  
MIN(ShippedDate) AS FechaMin, MAX(ShippedDate) AS FechaMAX,  
SUM(Freight) AS PesoTotal, AVG(Freight) AS PesoPromedio  
FROM Orders  
GROUP BY EmployeeID
```

IMPORTANTE

Es muy importante tener en cuenta que cuando utilizamos la cláusula **GROUP BY**, los únicos campos que podemos incluir en el **SELECT** sin que estén dentro de una función de agregación, son los que vayan especificados en el **GROUP BY**.

La cláusula **HAVING** se utiliza para filtrar grupos después de aplicar funciones de agregación. Es como usar un **WHERE** pero en vez de afectar a las filas de la tabla, afecta a los grupos obtenidos.

```
SELECT nombre_dept, AVG(sueldo) AS med_sueldo
FROM profesor
GROUP BY nombre_dept
HAVING AVG(sueldo) > 42000;
```

En este caso, `HAVING` aplica el filtro después de calcular la media para cada grupo.

6.10 Subconsultas anidadas

Las consultas válidas en SQL siempre generan una tabla que puede ser usada dentro de otra consulta SQL, fundamentalmente en las cláusulas `from` y `where` para comparaciones y comprobaciones entre otras tareas.

Esta posibilidad de **anidar** varios niveles de consultas permite la estructura de consultas muy sofisticadas y complejas en SQL, es la parte más rica del lenguaje pero también la que más práctica necesita para llegar a ser dominada.

6.10.1 Pertenencia a conjuntos

La conectiva `IN` permite comprobar si un valor pertenece a un conjunto devuelto por una subconsulta. Esto es útil para consultas en las que queremos verificar si una tupla cumple ciertas condiciones basadas en otro conjunto de resultados.

```
SELECT DISTINCT asignatura_id
FROM sección
WHERE semestre = 'Otoño' AND año = 2009
AND asignatura_id IN (
    SELECT asignatura_id
    FROM sección
    WHERE semestre = 'Primavera' AND año = 2010
);
```

Para encontrar todas las asignaturas que se enseñaron en otoño de 2009, pero no en primavera de 2010, podemos usar `NOT IN`:

```
SELECT DISTINCT asignatura_id
FROM sección
WHERE semestre = 'Otoño' AND año = 2009
AND asignatura_id NOT IN (
    SELECT asignatura_id
    FROM sección
```

```
WHERE semestre = 'Primavera' AND año = 2010
);
```

Podemos usar `IN` con valores específicos, por ejemplo, para encontrar a los profesores que no son ni "Mozart" ni "Einstein":

```
SELECT DISTINCT nombre
FROM profesor
WHERE nombre NOT IN ('Mozart', 'Einstein');
```

6.10.2 Comparación de conjuntos

SQL permite comparar un valor con los elementos de un conjunto devuelto por una subconsulta. Para esto se usan operadores como `> SOME` y `> ALL`.

```
SELECT nombre
FROM profesor
WHERE sueldo > SOME (
    SELECT sueldo
    FROM profesor
    WHERE nombre_dept = 'Biología'
);
```

Aquí, la subconsulta devuelve los sueldos de los profesores del departamento de Biología. La condición `> SOME` se cumple si el sueldo del profesor es mayor que al menos uno de esos sueldos.

Si quisiéramos encontrar los profesores cuyo sueldo es mayor que el de **todos** los profesores de Biología, usaríamos `> ALL`:

```
SELECT nombre
FROM profesor
WHERE sueldo > ALL (
    SELECT sueldo
    FROM profesor
    WHERE nombre_dept = 'Biología'
);
```

El operador `EXISTS` permite comprobar si una subconsulta devuelve alguna tupla. Esto es útil para verificar si un conjunto no está vacío.

```

SELECT asignatura_id
FROM sección AS S
WHERE semestre = 'Otoño' AND año = 2009
  AND EXISTS (
    SELECT *
    FROM sección AS T
    WHERE semestre = 'Primavera' AND año = 2010
      AND S.asignatura_id = T.asignatura_id
  );

```

El constructor `UNIQUE` permite verificar que una subconsulta no contenga tuplas duplicadas.

```

SELECT T.asignatura_id
FROM asignatura AS T
WHERE UNIQUE (
  SELECT R.asignatura_id
  FROM sección AS R
  WHERE T.asignatura_id = R.asignatura_id
    AND R.año = 2009
);

```

Aquí, `UNIQUE` garantiza que no haya duplicados en las asignaturas ofrecidas en 2009.

6.11 Expresiones de reunión en SQL

6.11.1 Condiciones de reunión

La operación de **unión natural** opera con dos relaciones y genera como resultado una relación con los pares de tuplas que tienen los mismos valores en los atributos que aparecen en los esquemas de ambas relaciones. Se utiliza la cláusula `NATURAL JOIN`.

```

select atrib1,...,atribN
from tabla1 natural join tabla2
where condición;

```

La cláusula `JOIN ... USING` permite indicar el nombre de los atributos a comparar:

```

SELECT atrib1,...,atribN
FROM (r1 natural join r2) join relación using (atrib)

```

La condición `ON` permite la reunión de un predicado general sobre relaciones. A diferencia de natural join el resultado contiene los atributos comparados dos veces en la reunión (una por cada tabla).

```
select *  
from r1 join r2 on r1.an=r2.an*
```

6.11.2 Reunión externa

La operación de reunión externa funciona de forma similar a la operación de reunión, creando tuplas en el resultado con valores nulos en los atributos de la relación que sólo aparecen en el esquema de una de las relaciones.

1. **Reunión externa izquierda (left outer join):** Conserva todas las filas de la tabla a la izquierda de la operación, incluso si no tienen coincidencias en la tabla de la derecha. Los valores de las columnas de la tabla de la derecha se rellenan con `null` en estas filas.

```
select *  
from estudiante  
left outer join matricula  
on estudiante.ID = matricula.ID;
```

2. **Reunión externa derecha (right outer join):** Es similar a la reunión externa izquierda, pero conserva todas las filas de la tabla a la derecha de la operación.

```
select *  
from matricula  
right outer join estudiante  
on estudiante.ID = matricula.ID;
```

3. **Reunión externa completa (full outer join):** Combina los resultados de la reunión externa izquierda y derecha, conservando todas las filas de ambas tablas. Las filas que no tienen coincidencias en la otra tabla tendrán valores `null` en las columnas correspondientes.

```
select *  
from estudiante  
full outer join matricula  
on estudiante.ID = matricula.ID;
```

El tipo de reunion por defecto en la **reunión interna**.

7. Structured Query Language (SQL) Otros Elementos

Escrito por **Adrián Quiroga Linares**.

7.1 Transacciones

En bases de datos, una **transacción** es una secuencia de operaciones SQL que se ejecutan como una unidad de trabajo. Las transacciones garantizan que las operaciones sean **atómicas** (o se completan todas, o ninguna) y preservan la **consistencia** de la base de datos.

1. **Inicio de la Transacción:** En SQL, una transacción se inicia automáticamente al ejecutar una sentencia SQL. No es necesario un comando específico para iniciar una transacción; cualquier consulta de actualización (como `UPDATE`, `INSERT`, o `DELETE`) marca implícitamente el comienzo.
2. **Finalización de la Transacción:** Para finalizar una transacción, SQL ofrece dos comandos principales:
 - `COMMIT`: Guarda todos los cambios realizados durante la transacción y los hace permanentes en la base de datos. Se garantiza que todos los cambios sean visibles para otros usuarios.
 - `ROLLBACK`: Revierte todos los cambios de la transacción, dejando la base de datos en el estado en el que se encontraba antes de iniciar la transacción. Esto es útil para deshacer operaciones en caso de errores.

Por ejemplo, si realizas varias operaciones en una aplicación bancaria para transferir fondos entre dos cuentas:

```
BEGIN; -- (Iniciado implícitamente al realizar la primera operación)
UPDATE cuentas SET saldo = saldo - 100 WHERE cuenta_id = 1; -- Restar dinero
UPDATE cuentas SET saldo = saldo + 100 WHERE cuenta_id = 2; -- Sumar dinero
COMMIT; -- Guardar los cambios
```

Si hay algún error entre las actualizaciones (por ejemplo, si se retira dinero pero falla al sumarlo), el `ROLLBACK` garantiza que la base de datos regrese al estado inicial, evitando inconsistencias.

3. **Fallos y Recuperación Automática:** Si ocurre un fallo (por ejemplo, una caída de sistema) antes de ejecutar `COMMIT`, SQL garantiza que los efectos de la transacción incompleta se reviertan automáticamente. Al reiniciar, el sistema restaura la base de datos al estado anterior de la transacción fallida.

7.2 Extensiones a la creación de tablas

Crear una tabla a partir de otra existente La sentencia `CREATE TABLE ... LIKE` se utiliza para crear una nueva tabla que tiene el mismo esquema (estructura de columnas y tipos de datos) que una tabla existente. Por ejemplo:

```
CREATE TABLE temp_profesor LIKE profesor;
```

En este caso, `temp_profesor` se crea con la misma estructura que la tabla `profesor`, pero sin ningún dato.

Crear una tabla a partir de una consulta

La sentencia `CREATE TABLE ... AS` permite crear una nueva tabla y llenarla con los resultados de una consulta. Por ejemplo:

```
CREATE TABLE t1 AS
SELECT * FROM profesor WHERE nombre_dept = 'Música' WITH DATA;
```

En este caso, `t1` se crea y se llena con todas las filas de la tabla `profesor` donde la columna `nombre_dept` tiene el valor 'Música'. La cláusula `WITH DATA` indica que la tabla se rellena con los resultados de la consulta. Si omites `WITH DATA`, según la norma SQL:2003, la tabla se crea vacía:

```
CREATE TABLE t2 AS
SELECT * FROM profesor WHERE nombre_dept = 'Música';
```

7.3 Autorización de privilegios

La autorización en bases de datos implica asignar diferentes tipos de permisos a los usuarios sobre las partes de la base de datos. Estos permisos se conocen como **privilegios** y se clasifican principalmente en cuatro tipos:

1. **Autorización de lectura de datos** (SELECT)
2. **Autorización de inserción de nuevos datos** (INSERT)
3. **Autorización de actualización de datos** (UPDATE)
4. **Autorización de borrado de datos** (DELETE)

- **Lectura:** Un usuario llamado `profesor1` puede consultar datos de la tabla `estudiantes`:

```
GRANT SELECT ON estudiantes TO profesor1;
```

- **Inserción:** Un usuario llamado `secretaria` puede agregar nuevos registros a la tabla:

```
GRANT INSERT ON estudiantes TO secretaria;
```

- **Actualización:** Un usuario llamado `profesor2` puede actualizar el correo electrónico de los estudiantes:

```
GRANT UPDATE (email) ON estudiantes TO profesor2;
```

- **Borrado:** Un usuario llamado `admin` puede eliminar registros de la tabla:

```
GRANT DELETE ON estudiantes TO admin;
```

7.4. Revocación de Privilegios

Los privilegios se conceden utilizando la sentencia `GRANT` y se revocan utilizando la sentencia `REVOKE`. Por ejemplo:

- Para conceder múltiples privilegios a un usuario:

```
GRANT SELECT, INSERT, UPDATE ON estudiantes TO profesor1;
```

- Para revocar un privilegio:

```
REVOKE UPDATE ON estudiantes FROM profesor2;
```

7.5 Roles

Los **roles** son grupos de privilegios que se pueden asignar a usuarios. Esto es útil cuando varios usuarios requieren los mismos permisos. Por ejemplo, en una universidad, podríamos tener roles como `profesor`, `estudiante`, o `administrador`.

- **Creación de un rol:**

```
CREATE ROLE profesor;
```

- **Concesión de privilegios al rol:**

```
GRANT SELECT ON estudiantes TO profesor;
```

- **Asignación de roles a usuarios:**


```
GRANT profesor TO profesor1;
```

Cuando un usuario se conecta a la base de datos, tiene todos los privilegios que se le han concedido directamente, así como los que hereda de cualquier rol que se le haya asignado.

8. Normalización del Modelo Relacional

Escrito por **Adrián Quiroga Linares**.

8.1 Introducción

La normalización del modelo relacional de una base de datos tiene una fuerte **base matemática** que se apoya en el concepto de dependencia funcional de atributos. EL descubrimiento de dependencias funcionales entre atributos supone la representación de la semántica que se incorpora a la base de datos, se podría decir que es equivalente a descubrir conjuntos de entidades y relaciones en el MER.

Una vez representada la semántica mediante las dependencias funcionales el proceso de normalización consiste en el cumplimiento de determinadas condiciones denominadas **formas normales**.

A veces es necesario descomponer los esquemas en tablas más pequeñas siendo válidas, únicamente las descomposiciones reversibles, sin pérdidas, de forma que si se unen las tablas de vuelve a tener la original.

Conclusión: Un buen diseño de base de datos relacional evita la redundancia, asegura la consistencia, y facilita la representación precisa de datos independientemente.

La **Dependencia Funcional** es la base de la normalización. Un conjunto de atributos determina funcionalmente a otro si, para todas las tuplas de una relación, cuando los valores del primero se repiten, entonces los valores del segundo se repiten.

Las diversas formas normales van imponiendo sucesivas condiciones a las relaciones que forman la base de datos.

8.2 Primera Forma Normal

Cuando una tabla contiene únicamente atributos **atómicos**, se dice que está en la **Primera Forma Normal**. En una base de datos del MR esta primera forma es la única **obligatoria**, el resto de formas normales se limitan a mejorar el diseño evitando los problemas que podrían aparecer.

Está prohibido tener **atributos compuestos y multivalorados**.

8.3 Segunda Forma Normal (2FN)

Impide la existencia de **dependencias funcionales parciales** (*en las dependencias funcionales parciales el lado izquierdo es una parte de una superclave pero no la superclave completa*).

8.4 Tercera Forma Normal (3FN)

Impide la existencia de **dependencias funcionales transistivas** (en las dependencias funcionales transistivas el lado izquierdo no forma parte de una superclave).

8.5 Forma Normal de Boyce-Codd (FNBC)

Obliga a que en todas las **dependencias funcionales del lado izquierdo** sea una superclave de la relación. Si en el **MER** tenemos un entidad débil nunca puede estar en FNBC.

Cuando se encuentra alguna relación con alguna dependencia funcional que no cumple las condiciones se **procede a descomponerla** (habitualmente en 2). Hay que escoger descomposiciones de reunión sin pérdidas que preserven las dependencias.

Por encima de la **FNBC** existen otras formas normales que van imponiendo condiciones cada vez más restrictivas, pero normalmente no se usan en la práctica. El objetivo habitual de la normalización es conseguir que todas las **relaciones esten en FNBC**.

8.6 Teoría de las Dependencias Funcionales

Una **dependencia funcional** entre dos conjuntos de atributos X y Y en una relación R se denota como:

$$X \rightarrow Y$$

Esto significa que si dos tuplas (*filas*) tienen los mismos valores para el conjunto de atributos X , entonces también deben tener los mismos valores para el conjunto Y . Es decir, el valor de X determina el valor de Y .

8.6.1 Cierre de un conjunto de dependencias funcionales

Cuando hablamos del **cierre de un conjunto de dependencias funcionales** F , denotado como F^+ , nos referimos al conjunto de todas las dependencias funcionales que se pueden derivar lógicamente a partir de las dependencias en F . El cierre es importante porque, cuando estamos comprobando si un esquema está en una forma normal (como la FNBC o la 3FN), no basta con mirar únicamente las dependencias que se nos dan. Debemos considerar todas las dependencias funcionales que pueden deducirse de ellas.

Ejemplo: Supongamos que tenemos el siguiente conjunto de dependencias funcionales para un esquema de relación $R(A, B, C, G, H, I)$:

$$F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$$

Queremos determinar si $A \rightarrow H$ se puede deducir a partir de F .

1. Sabemos que $A \rightarrow B$, lo que implica que si dos tuplas tienen el mismo valor para A , entonces también deben tener el mismo valor para B .
2. También sabemos que $B \rightarrow H$, lo que implica que si dos tuplas tienen el mismo valor para B , entonces deben tener el mismo valor para H .
3. Si $t_1[A] = t_2[A]$, entonces $t_1[B] = t_2[B]$ por $A \rightarrow B$.
4. Luego, $t_1[B] = t_2[B]$ implica que $t_1[H] = t_2[H]$ por $B \rightarrow H$.

Por lo tanto, hemos demostrado que $A \rightarrow H$ se cumple, lo que significa que $A \rightarrow H$ **está implicado lógicamente por F** .

8.6.2 Axiomas de Armstrong

Los **axiomas de Armstrong** y sus reglas adicionales permiten descubrir nuevas dependencias funcionales a partir de un conjunto de dependencias dados.

1. **Reflexividad:** Si $\alpha \subseteq \beta$, entonces $\beta \rightarrow \alpha$.
2. **Aumentatividad:** Si $\alpha \rightarrow \beta$ y γ es un conjunto de atributos, entonces $\gamma\alpha \rightarrow \gamma\beta$.
3. **Transitividad:** Si $\alpha \rightarrow \beta$ y $\beta \rightarrow \gamma$, entonces $\alpha \rightarrow \gamma$.

Otras reglas útiles

Además de los axiomas de Armstrong, existen reglas adicionales que se usan para derivar dependencias funcionales más fácilmente:

- **Unión:** Si $\alpha \rightarrow \beta$ y $\alpha \rightarrow \gamma$, entonces $\alpha \rightarrow \beta\gamma$.
- **Descomposición:** Si $\alpha \rightarrow \beta\gamma$, entonces $\alpha \rightarrow \beta$ y $\alpha \rightarrow \gamma$.
- **Pseudotransitividad:** Si $\alpha \rightarrow \beta$ y $\gamma\beta \rightarrow \delta$, entonces $\alpha\gamma \rightarrow \delta$.

8.7 Recubrimiento Canónico

El **recubrimiento canónico** es el conjunto mínimo equivalente de dependencias funcionales. Dos conjuntos son equivalentes si sus cierres son iguales.

8.8 Pasos para la normalización

TIPs: Está en Boyce-Codd si todos los determinantes del recubrimiento canónico ($a \rightarrow B$, a es el *determinante*) forman parte de la clave candidata calculada con el cierre. **Si no están en Boyce-Codd**, localizamos atributos equivalentes ($a \rightarrow B$ y $B \rightarrow a$), si pasa esto se llevan todos los atributos equivalentes a una nueva relación y en la relación universal (U) se deja solo uno de esos atributos. **Después**, si sigue sin cumplir alguna relación $a \rightarrow B$ se crea una nueva tabla con los atributos a, B y la tabla original se elimina B .

Si ponen este tremendo cigarro, para sacar las dependencias funcionales, pensar las multiplicidades que se darían. Si son del tipo 1 a 1 o N a N se determinan mutuamente, si son 1..N o N..1 el lado N determina al lado 1.

IMPORTANTE

Normalización: Técnica para producir un conjunto de relaciones con una serie de propiedades deseables a partir de unos requisitos de datos.

1FN: Relación en la que la intersección de toda fila y columna contiene un valor atómico

Dependencia Funcional: Describe la relación existente entre atributos de una relación. Si A y B son conjuntos de atributos de la relación R, B será funcionalmente dependiente de A ($A \rightarrow B$) si cada valor de A está asociado con exactamente un valor de B.

Determinante: Grupo de atributos del lado izquierdo de una Dependencia Funcional.

Dependencia Funcional Completa: Si A y B son conjuntos de atributos de la relación R, B depende funcionalmente de manera completa de A si B depende funcionalmente de A pero no de ningún subconjunto de A.

Dependencia Parcial: Dependencia funcional en la cual existe algún atributo que puede eliminarse del determinante y la dependencia continúa verificándose

Dependencia Transitiva: Si A, B y C son conjuntos de atributos de la relación R, C depende transitivamente de A a través de B si C depende funcionalmente de B y B depende funcionalmente de A.

2FN: Relación que está en 1FN y en la que todo atributo que no sea de una clave candidata depende funcionalmente de forma completa de cualquier clave candidata.

3FN: Relación que está en 2FN y en la que ningún atributo que no sea de una clave candidata depende transitivamente de cualquier clave candidata

FNBC: Relación en la que el determinante de todas las dependencias funcionales es una clave candidata.

Dependencia Multivaluada: Dependencia entre conjuntos de atributos A, B y C con valores independientes de modo que para cada valor de A hay un conjunto de valores de B y un conjunto de valores de C.

4FN: Relación que está en FNBC y no contiene dependencias multivaluadas.

5FN: Relación que no presenta dependencias de combinación (propiedad de la descomposición de relaciones que garantiza que no se generan tuplas incorrectas al volver a combinar relaciones de una descomposición mediante una operación de reunión natural)

9. Sistemas de Bases de Datos

Un **sistema gestor de bases de datos (SGBD)** consiste en una colección de datos interrelacionados y un conjunto de programas para acceder a dichos datos.

El **objetivo** de un SGBD es proporcionar una forma de almacenar y recuperar la información y la provisión de mecanismos para su manipulación.

9.1 Inconvenientes a evitar en un SGBD

- **Redundancia e inconsistencia de los datos**
- **Dificultad en el acceso a los datos**
- **Aislamiento de datos**
- **Problemas de integridad**
- **Problemas de atomicidad**
- **Anomalías en el acceso concurrente**
- **Problemas de seguridad**

El objetivo de los **SGBD** es paliar los inconvenientes que tienen los sistemas de procesamiento de archivos.

9.2 Visión de los datos

Una de las principales finalidades de los sistemas de bases de datos es ofrecer a los usuarios una **visión abstracta de los datos**.

Niveles de abstracción:

- **Nivel físico:** describe cómo se almacenan los datos
- **Nivel lógico:** describe qué datos se almacenan en la BD y qué relaciones existen entre ellos. Este nivel es el que usan los administradores de bases de datos.
- **Nivel de vistas:** este nivel solo muestra la parte de la BD, sirve para simplificar la interacción de los usuarios con el sistema.

La colección de información almacenada en una base de datos en un momento dado se denomina **ejemplar** de la BD. El diseño general de la BD se denomina **esquema** de la BD.

El **esquema físico** describe el diseño a nivel físico de la BD, el **esquema lógico** describe a su diseño a nivel lógico. Una BD también puede tener varios esquemas a nivel de vistas denominados **subesquemas**, que describen varias vistas de la BD.

El esquema lógico es el más importante.

Modelo de datos Ofrece un modo de describir el diseño de las BD en los tres niveles. Pueden clasificarse en cuatro categorías diferentes:

- **Modelo entidad-relación**
- **Modelo relacional**
- **Modelo de datos semiestructurados** (*elementos del mismo tipo pueden tener atributos diferentes*)
- **Modelo de datos basado en objetos** (*extensión del MER con encapsulación*).

Para conseguir el objetivo de la visión abstracta, en primer lugar se establecen los 3 niveles independientes separando los aspectos de diseño de los de implementación. En segundo lugar se establecen los esquemas de estos niveles y, por último, se establece un modelo de datos que ofrezca una colección de herramientas conceptuales para definir los esquemas de los diferentes niveles.

9.3 Lenguajes de bases de datos

Es un lenguaje que permite a los usuarios tener acceso a los datos organizados mediante el modelo de datos elegido, o manipularlos.

El **lenguaje de definición de datos** expresa los esquemas definidos en un determinado modelo de datos. En el se definen las restricciones de integridad:

- **Restricción de dominio:** valores que puede tomar cada atributo
- **Integridad referencial**
- **Asertos:** condiciones que la BD debe satisfacer siempre
- **Autorización:** de lectura, de inserción, de actualización o de eliminación

El **lenguaje de manipulación de datos** permite consultar a la BD para extraer información y explotar la información introducida en ella. Un LMD puede ser procedimental o declarativo.

Una **consulta** es una instrucción para recuperar la información, la parte del LMD implicada en la recuperación de información se denomina **lenguaje de consultas**.

[Conceptos] **Bases de datos relacionales**-> se basan en el modelo relacional y usan un conjunto de tablas que representan los datos y las relaciones En la fase de **diseño conceptual** de una BD, el diseñador escoge el modelo de datos y traduce los requisitos en un esquema conceptual de la BD. **Modelo Entidad-Relación** -> se basa en un conjunto de objetos denominados entidades y las relaciones que existen entre ellos **Normalización**-> se basa en un conjunto de objetos denominados entidades y las relaciones que existen entre ellos.

9.4 Gestor de almacenamiento

Es un módulo de programa que proporciona la interfaz entre los datos de bajo nivel almacenados en la BD y los programas de aplicación y las consultas.

Componentes del gestor:

- **Gestor de autorizaciones e integridad**
- **Gestor de transacciones**
- **Gestor de archivos**
- **Gestor de la memoria intermedia**

Estructuras de datos del gestor:

- **Archivos de datos**
- **Diccionario de datos**
- **Índices**

9.5 Gestión de transacciones

Requisitos:

- **Atomicidad:** una transacción debe producirse por completo o no producirse en absoluto
- **Consistencia:** es esencial que las transacciones preserven la consistencia
- **Durabilidad:** los valores nuevos tras la transacción deben persistir aunque haya un fallo en el sistema Una **transacción** es un conjunto de operaciones que lleva a cabo una única función lógica en una aplicación de bases de datos

El **componente de gestión de transacciones** es el responsable de garantizar las propiedades de durabilidad y atomicidad. Es responsabilidad del **gestor de control de concurrencia** controlar la interacción de las transacciones concurrentes para garantizar la consistencia.

El término **minería de datos** se refiere, en líneas generales, al proceso de análisis semiautomático de grandes bases de datos para descubrir patrones útiles, intenta descubrir reglas y patrones en los datos.